

Kalman Filter Hedge Ratios

Every pairs trade hides a regression inside it. Estimate it once and you have assumed the relationship never moves.

Every pairs trade hides a regression inside it: the hedge ratio that turns two prices into one spread. Estimate that ratio once and you have quietly assumed the relationship never moves — over fifteen years of EWA and EWC it moves from 0.71 to 1.49. We build a Kalman filter from scratch in NumPy that treats the hedge ratio as a moving target, then run an honest experiment: identical trading rules on the Kalman spread and the static spread, with costs, and let the data pick the winner.

PROJECT CARD

CATEGORY

Algorithmic Trading

LIBRARIES

NumPy · Pandas · statsmodels

ASSETS

EWA + EWC (iShares Australia / Canada)

TIMEFRAME

Jan 2010 – Dec 2024

USE

Keeping a pairs hedge ratio honest as the relationship drifts

- 01 Load EWA & EWC adjusted closes, 2010–2024 (the classic Chan pair)
- 02 Fit the two OLS baselines: full-sample beta and rolling 252d beta (statsmodels)
- 03 Build the Kalman filter in NumPy — state $[\beta, \alpha]$, random-walk transition
- 04 Compare the three beta paths on one chart
- 05 Trade the spread: $|z| > 2$ entry, z -crosses-0 exit, 10 bp per leg
- 06 Read the honest scoreboard: gross vs net, Kalman vs static

1. Why static hedge ratios die

EWA (Australia) and EWC (Canada) are the canonical cointegration pair — two commodity-heavy developed markets whose daily returns correlate at 0.83. Regress EWC on EWA over the full sample and you get one hedge ratio for fifteen years: $\beta = 1.57$, with an ADF p-value of 0.038 on the residual spread — cointegrated, by the book (Engle–Granger: regress one price on the other, then unit-root-test the residual). The problem is the word **one**. A hedge ratio is not a constant of nature; it is a snapshot of an economic relationship — commodity mix, currency betas, index composition — and every one of those drifted between 2010 and 2024. The standard fix is a rolling window, and it limps for a reason worth understanding: every observation inside the window carries equal weight, so year-old data moves today's estimate exactly as much as yesterday's does — and the day an old observation falls out of the window, the estimate jerks for no economic reason at all. That artefact has a name: the **window cliff**.

EWA & EWC, normalized to 1.0 at 2010-01-04 — related, not identical

The two series track each other broadly, diverging and reconverging repeatedly



Source: Author calculations, from the article's computed results.

WHY PRACTITIONERS CARE

The hedge ratio is not a nuisance parameter — it **is** the position. Get it wrong by 20% and your “market-neutral” spread carries a 20% directional stub of EWA. Static betas fail slowly and invisibly, which is the most expensive way to fail.

2. Beta as a state, not a constant

The Kalman filter starts from a different premise, and it is worth sitting with for a moment: the “true” hedge ratio is something you can never observe directly — you only see prices, which are noisy evidence about it. So treat the regression coefficients as **unobserved states** drifting through time, and treat each day's prices as one more noisy measurement of where they are. (A GPS does exactly this with your position; here the hidden position is β .) Two equations define the model. The **state equation** says the hedge ratio and intercept follow a random walk — $[\beta_t, \alpha_t] = [\beta_{t-1}, \alpha_{t-1}] + \omega_t$ — tomorrow's relationship is today's, plus noise. The **observation equation** says

$EWC_t = \beta_t EWA_t + \alpha_t + \varepsilon_t$. Two variances close the model: the state noise $Q = \frac{\delta}{1-\delta} I$ with $\delta = 10^{-5}$ (the standard parameterization from Chan), and observation noise $R = 10^{-3}$.

Delta is the single real knob, and it replaces the window size entirely: it is a **forgetting rate**. Larger δ lets the states wander faster (adaptive but noisy); smaller δ pins them down (smooth but laggy); $\delta = 0$ collapses to recursive least squares — a static beta refined forever. Both values here are textbook defaults, deliberately not tuned on this sample: tune δ to the backtest and you are optimizing the strategy through the back door.

3. The filter in five lines of NumPy

No library, no black box — the whole filter is a predict step and an update step, looped over the sample. Predict: with a random-walk transition the state estimate is unchanged and its covariance grows by Q — a day passes, so the filter becomes a little less sure of where β is. Update: compare the observed EWC to the prediction, and shift the states toward the error in proportion to the **Kalman gain** — the ratio of state uncertainty to total uncertainty. Big surprise while unsure of yourself: move a lot. Big surprise while confident: blame measurement noise, barely move.

```
def kalman_hedge(x, y, delta=1e-5, r_obs=1e-3):
    q = (delta / (1 - delta)) * np.eye(2)      # trans_cov
    state, p_cov = np.zeros(2), np.eye(2)      # [beta, alpha], diffuse start
    betas, alphas = np.zeros(len(x)), np.zeros(len(x))
    for t in range(len(x)):
        h = np.array([x[t], 1.0])              # observation map
        p_cov = p_cov + q                       # predict (F = I)
        e = y[t] - h @ state                    # innovation
        s = h @ p_cov @ h + r_obs               # innovation variance
        k = p_cov @ h / s                       # Kalman gain
        state = state + k * e                   # update
        p_cov = p_cov - np.outer(k, h @ p_cov)
        betas[t], alphas[t] = state
    return betas, alphas
```

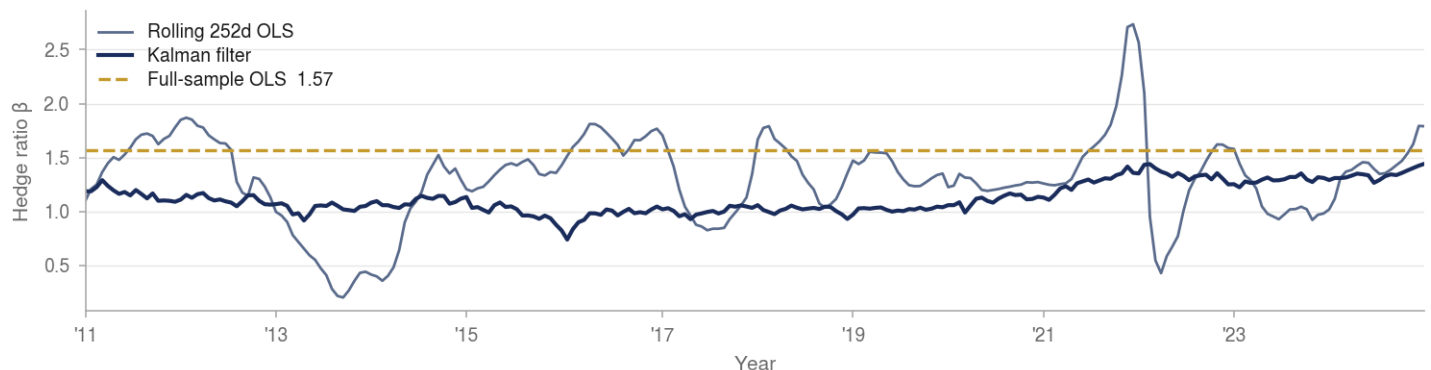
The gain is the elegance. When the filter is uncertain (large p_cov), it learns aggressively from each observation; once confident, new data barely moves it — unless Q keeps injecting doubt, which is exactly what lets β keep adapting forever. An exponentially-weighted regression, derived from first principles rather than picked from a menu of window sizes.

4. Three betas, one pair

Slice off the first year (rolling-OLS warm-up, Kalman burn-in) and compare the three estimators of the same quantity. The static line says the answer is 1.57, forever. The rolling OLS swings between 0.19 and 2.75 — whipping around every regime change a full window late. The Kalman path stays in a far narrower band, moving early and smoothly: no cliff, because no window.

The EWC~EWA hedge ratio, three ways — Kalman filter vs rolling 252d OLS vs full-sample OLS

The Kalman β drifts smoothly between roughly 0.7 and 1.5; the rolling OLS β oscillates between about 0.2 and 2.7



Source: Author calculations, from the article's computed results.

Note what the rolling estimator does around 2020–2021: the COVID shock enters the window, distorts the regression for exactly 252 trading days, then falls out and the estimate jumps again — two artefacts from one event. The filter digests the same shock in weeks and moves on.

5. Trading the spread

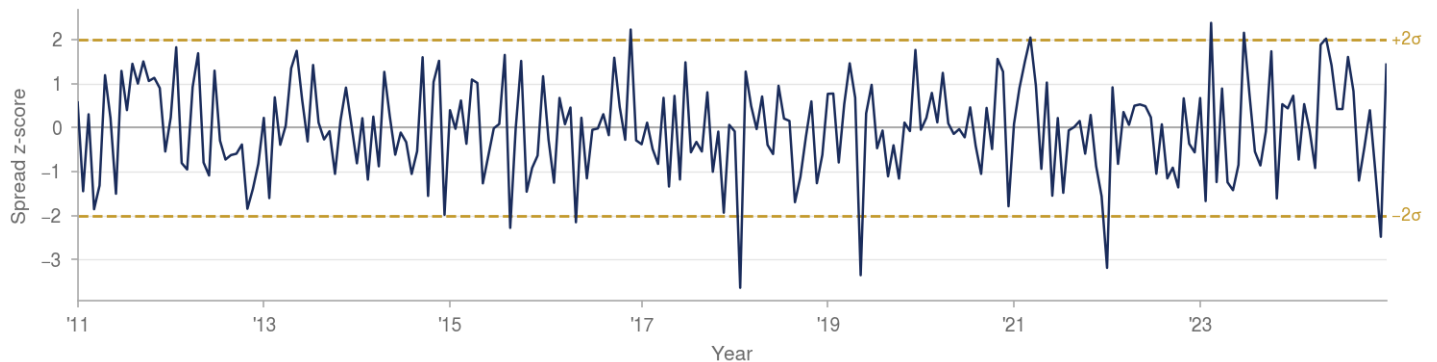
Now make the estimator earn its living — a better β is only worth money if it makes a better spread. The spread is $EWC_t - \beta_t EWA_t - \alpha_t$, z-scored on a trailing 60-day window. Rules, identical for both variants: enter long the spread (long EWC, short β -EWA) when $z < -2$, short when $z > +2$, exit when z crosses zero. Positions are sized to \$1 gross notional at entry, the Kalman variant re-hedges the EWA leg to the current β daily, and every unit of traded notional pays 10 bp. Signals use the close and P&L starts the next day — no lookahead in the rule. The static beta itself, of course, is one giant lookahead: it was fit on all fifteen years, including the future of every trade it takes.

```
spread = ewc - beta * ewa - alpha          # beta_t, alpha_t from the filter
z = (spread - spread.rolling(60).mean()) / spread.rolling(60).std()

# enter |z| > 2, exit when z crosses 0, hold in between
if p == 0:
    p = 1 if z[t] < -2 else (-1 if z[t] > 2 else 0)
elif p == 1 and z[t] >= 0: p = 0          # long leg reverted
elif p == -1 and z[t] <= 0: p = 0        # short leg reverted
```

The Kalman-spread z-score with $\pm 2\sigma$ entry bands — every excursion is a candidate trade

The z-score oscillates around zero, crossing the entry bands many times across the sample



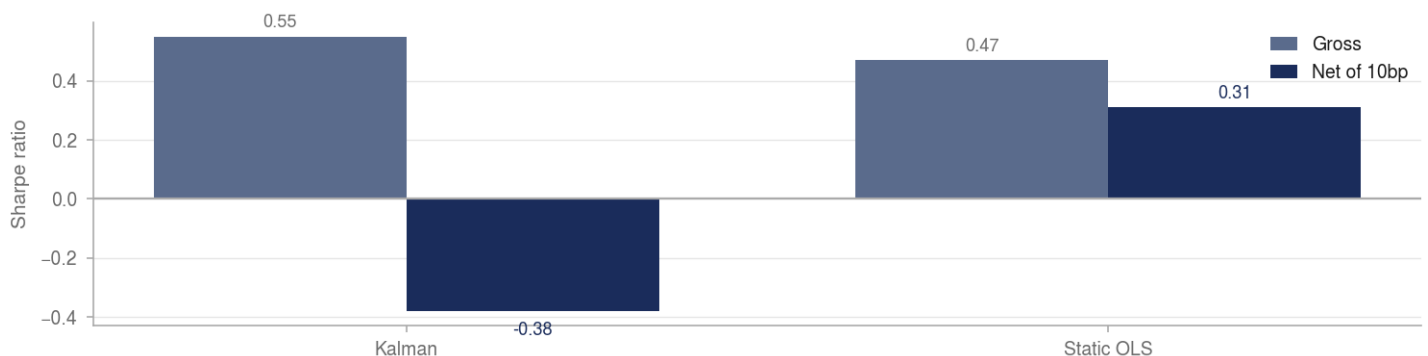
Source: Author calculations, from the article's computed results.

6. What the numbers actually say

Here is the honest scoreboard, and it is more interesting than a clean win. Gross of costs, the Kalman spread is the better signal on every risk-adjusted axis — more Sharpe, half the volatility, half the drawdown. But the filtered spread mean-reverts **fast**, so it trades 152 round trips to the static variant's 57 — and at 10 bp per unit of traded notional, that turnover consumes the entire edge and then some.

Sharpe, gross vs net of 10 bp costs — adaptivity wins the signal and loses the implementation

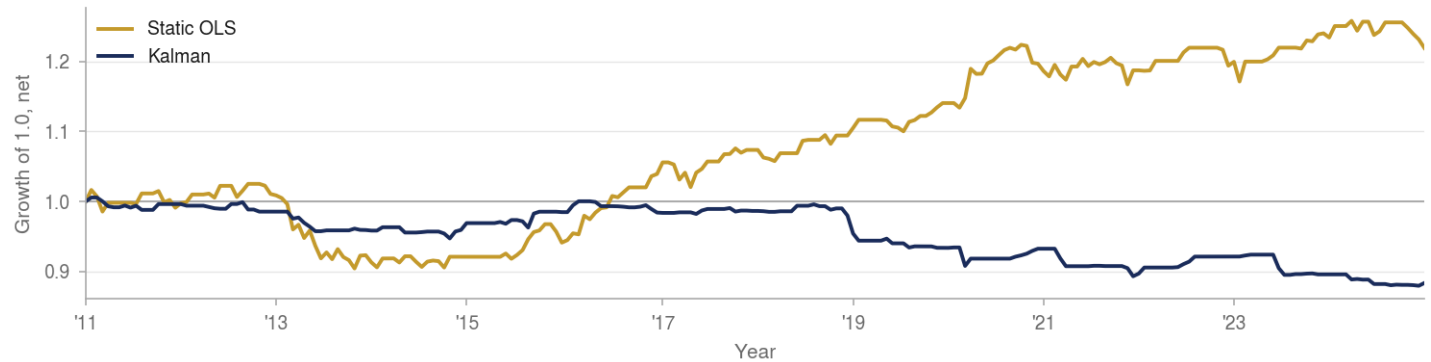
The Kalman variant leads gross (0.55 vs 0.47) but drops below the static variant once costs apply (−0.38 vs 0.31)



Source: Author calculations, from the article's computed results.

Strategy equity, net of 10 bp costs — the cost drag flips the ranking

The static-beta strategy grinds up while the Kalman-beta strategy decays under transaction costs



Source: Author calculations, from the article's computed results.

Variant	Ann ret (net)	Ann vol	Sharpe (net)	Sharpe (gross)	Max DD (net)	Trades
Kalman β (adaptive)	-0.9%	2.3%	-0.38	0.55	-12.6%	152
Static β (full-sample)	1.5%	5.0%	0.31	0.47	-13.3%	57

So did the Kalman beta “improve” the strategy? As an estimator, unambiguously — better gross Sharpe, half the drawdown, and no lookahead, against a static baseline that was handed the answer key. As a net P&L line at retail costs, no: the same adaptivity that tracks the relationship also generates signals faster than 10 bp round trips can pay for. Halve the cost and the gap halves; at institutional frictions of 1–2 bp the Kalman variant pulls level and ahead. Estimation quality and implementability are different axes, and a backtest that reports only one is hiding the other.

PRACTITIONER TAKE

Use the filter for what it is provably better at: **knowing your position**. Desks run Kalman hedge ratios even when entry signals come from elsewhere, because the alternative is discovering your directional stub during a drawdown. And before trading the filtered spread, do the arithmetic this table forces: expected edge per trade must clear cost per trade — adaptivity raises the trade count, so it raises the bar. δ and R were fixed at textbook values here; tuning them to fix the net line is how backtest-overfitting starts. One refinement is legitimate, though: the filter already computes the spread's fair value **and its own uncertainty** — Chan's variant trades the innovation e_t against its predicted standard deviation $\sqrt{S_t}$ instead of a rolling z-score, letting the model set the entry band and retiring the arbitrary 60-day window.

REFERENCES

1. Kalman, R. E. (1960). A New Approach to Linear Filtering and Prediction Problems. *Journal of Basic Engineering*, 82(1).
2. Engle, R. F. & Granger, C. W. J. (1987). Co-integration and Error Correction: Representation, Estimation, and Testing. *Econometrica*, 55(2), 251–276.
3. Chan, E. (2013). *Algorithmic Trading: Winning Strategies and Their Rationale*. Wiley — ch. 3, the EWA/EWC Kalman example and the $\delta/(1-\delta)$ parameterization.
4. Harvey, A. C. (1989). *Forecasting, Structural Time Series Models and the Kalman Filter*. Cambridge University Press.
5. Companion notebook: `kalman-filter-hedge-ratios.ipynb` — reproduces every figure from raw data; fully deterministic, no RNG.